Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

EXPERIMENT NO. 3

AIM:

To construct layout compositions using Next.js root layouts and localized sub-layouts to safely maintain state during client-side transitions.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:

- **Nested Layout Architecture (App Router):** Next.js App Router organizes user interfaces through dynamic segment nesting. A layout.tsx file wraps child pages (children) and persists across route navigations within its directory scope. Unlike standard page components, layouts do not unmount during navigation between sibling routes in the same subtree.
- **Partial Rendering & State Preservation:** During client-side routing via Next.js <Link> components, Next.js performs partial rendering. It evaluates and re-renders only the dynamic leaf segments (page.tsx) while keeping parent layouts mounted in React's component tree. Consequently, local React state (useState, useRef), active timers (useEffect), and media playback initialized within a localized sub-layout remain entirely uninterrupted across route transitions.
- **Layouts (**layout.tsx**) vs. Templates (**template.tsx**):** While layouts preserve state and maintain their existing DOM nodes during navigation, templates create a new component instance on every navigation pass. Replacing a layout.tsx with a template.tsx forces React to reset internal hook states and re-mount DOM elements on each route change.

STEPS / ALGORITHM:

Step 1: Create the Root Layout (app/layout.tsx**)**

Define the global document structure and top-level navigation container.

```
import Link from 'next/link';

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body style={{ fontFamily: 'system-ui, sans-serif', padding:
'20px', background: '#0a0a0a', color: '#ededed' }}>
        <nav style={{ marginBottom: '20px', display: 'flex', gap: '15px'
}}>
          <Link href="/" style={{ color: '#61dafb' }}>Home Root</Link>
          <Link href="/blog" style={{ color: '#61dafb' }}>Blog Sub-
Tree</Link>
        </nav>
        {children}
      </body>
    </html>
  );
}
```

Step 2: Define the Root Landing Page (app/page.tsx)

Build a standard landing page component at the root route /.

```
export default function HomePage() {
  return (
    <div>
      <h1>Root Landing Page</h1>
      <p>Navigate to the Blog Sub-Tree to test localized sub-
layout state preservation.</p>
    </div>
  );
}
```

Step 3: Implement Localized Sub-Layout with State (app/blog/layout.tsx)

Construct a client-rendered sub-layout ('use client') maintaining persistent uptime and counter state across nested /blog sub-routes.

```
'use client';

import { useState, useEffect } from 'react';
import Link from 'next/link';

export default function BlogLayout({
  children,
```

```
}: {
  children: React.ReactNode;
}) {
  const [counter, setCounter] = useState<number>(0);
  const [seconds, setSeconds] = useState<number>(0);

  // Active interval timer to prove layout persistence during
navigation
  useEffect(() => {
    const timer = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    return () => clearInterval(timer);
  }, []);

  return (
    <div style={{ border: '2px solid #2563eb', padding:
'20px', borderRadius: '8px', background: '#121212', color:
'#fff' }}>
      <div style={{ paddingBottom: '15px', marginBottom:
'15px', borderBottom: '1px solid #333' }}>
        <h2 style={{ color: '#60a5fa', margin: '0 0 10px 0'
}}>Localized Blog Sub-Layout</h2>

        <div style={{ display: 'flex', gap: '20px', margin:
'10px 0' }}>
          <p>Uptime Timer: <strong style={{ color: '#22c55e'
}}>{seconds}s</strong></p>
          <p>State Counter: <strong style={{ color: '#f59e0b'
}}>{counter}</strong></p>
        </div>

        <button
          onClick={() => setCounter((prev) => prev + 1)}
          style={{ padding: '8px 16px', background:
'#2563eb', color: '#fff', border: 'none', borderRadius:
'4px', cursor: 'pointer' }}
        >
          Increment State Counter
        </button>
```

```
        <nav style={{ marginTop: '15px', display: 'flex',
gap: '15px' }}>
          <Link href="/blog" style={{ color: '#93c5fd'
}}>Blog Index</Link>
          <Link href="/blog/post-1" style={{ color: '#93c5fd'
}}>Post 1</Link>
          <Link href="/blog/post-2" style={{ color: '#93c5fd'
}}>Post 2</Link>
        </nav>
      </div>

      <main>{children}</main>
    </div>
  );
}
```

Step 4: Build Blog Index View (app/blog/page.tsx**)**

Create the entry point page for the /blog route.

```
export default function BlogIndexPage() {
  return (
    <div>
      <h3>Blog Overview Page</h3>
      <p>Increment the counter or watch the uptime timer
above, then click between Post 1 and Post 2.</p>
    </div>
  );
}
```

Step 5: Implement Dynamic Sub-Page View (app/blog/[slug]/page.tsx**)**

Implement an asynchronous Server Component to handle dynamic post routes (/blog/post-1, /blog/post-2).

```
export default async function BlogPostPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;

  return (
    <div>
```

```
      <h3>Post Page: <span style={{ color: '#f59e0b'
}}>{slug}</span></h3>
      <p>The page content rendered dynamically, while the
outer timer and counter in the sub-layout remained
unchanged.</p>
    </div>
  );
}
```

Step 6: Run and Validate Layout State Retention

Execute the development server:

```
npm run dev
```

1. Navigate to http://localhost:3000/blog.
2. Click **"Increment State Counter"** to modify the internal state.
3. Switch between **Post 1**, **Post 2**, and **Blog Index** using client links. Observe that the timer continues incrementing and the counter holds its value without resetting.

OBSERVATION:

Observe how navigating between dynamic routes updates the inner viewport without triggering an unmount cycle on BlogLayout. The Uptime Timer interval continues ticking continuously, and the State Counter value is strictly preserved.

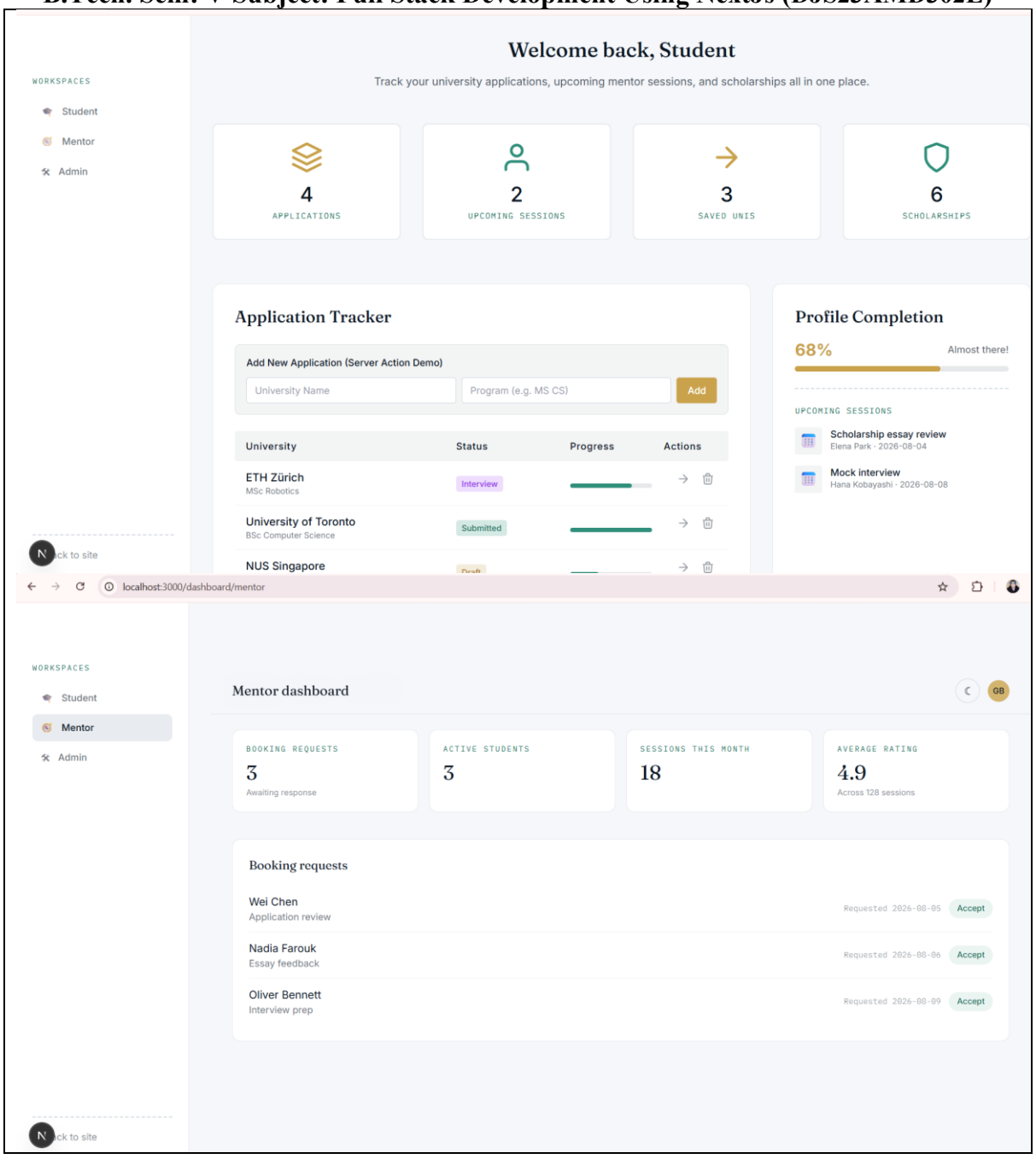
Note that navigating back to the root route (/) unmounts the localized sub-layout segment. Re-entering /blog instantiates a new sub-layout component instance, resetting the state variables back to their initial values (0).

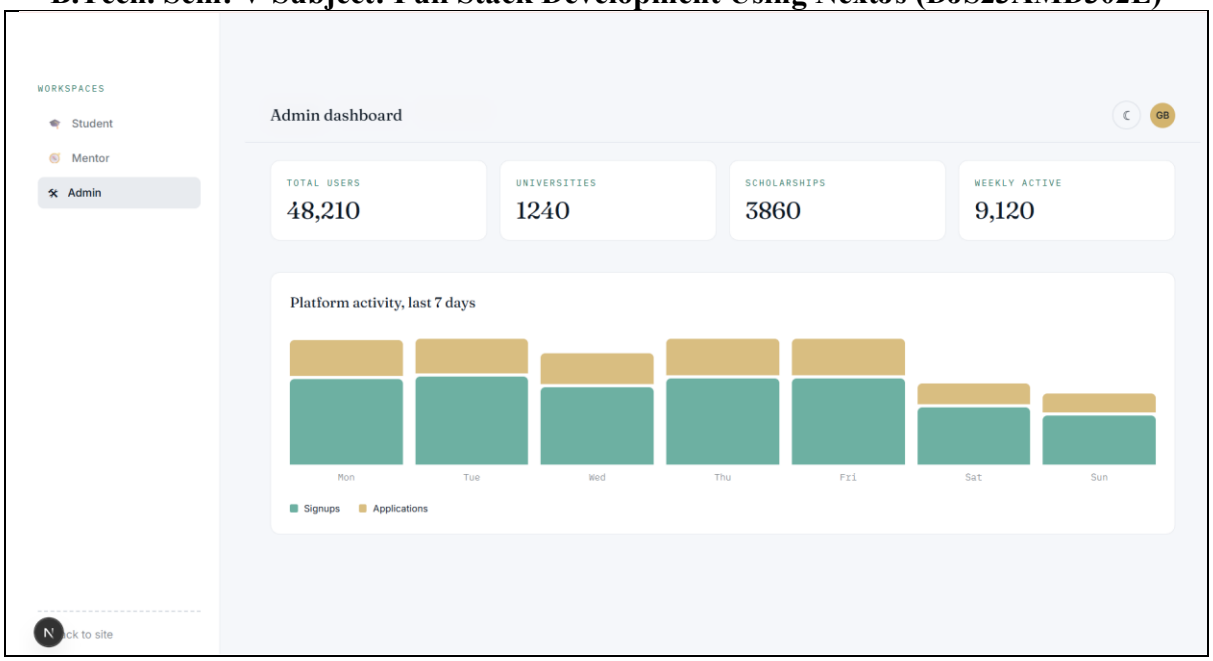
OUTPUT:

Development Terminal Execution:

```
▲ Next.js 16.x
  - Local:        http://localhost:3000
  - Environments: .env

 ✓ Starting...
 ✓ Ready in 980ms
 ○ Compiling /blog/[slug] ...
 ✓ Compiled /blog/[slug] in 240ms
 GET /blog/post-1 200 in 45ms
```


WORKSPACES
Student
Mentor
Admin
Welcome back, Student
Track your university applications, upcoming mentor sessions, and scholarships all in one place.
4
APPLICATIONS
2
UPCOMING SESSIONS
3
SAVED UNIS
6
SCHOLARSHIPS
Application Tracker
Add New Application (Server Action Demo)
University Name
Program (e.g. MS CS)
Add
University
Status
Progress
Actions
ETH Zürich
MSc Robotics
Interview
University of Toronto
BSc Computer Science
Submitted
NUS Singapore
Profile Completion
68%
Almost there!
UPCOMING SESSIONS
Scholarship essay review
Elena Park · 2026-08-04
Mock interview
Hana Kobayashi · 2026-08-08
localhost:3000/dashboard/mentor
WORKSPACES
Student
Mentor
Admin
Mentor dashboard
BOOKING REQUESTS
3
Awaiting response
ACTIVE STUDENTS
3
SESSIONS THIS MONTH
18
AVERAGE RATING
4.9
Across 128 sessions
Booking requests
Wei Chen
Application review
Requested 2026-08-05
Accept
Nadia Farouk
Essay feedback
Requested 2026-08-06
Accept
Oliver Bennett
Interview prep
Requested 2026-08-09
Accept



Rendered Viewport Verification:

- **Sibling Sub-Route Navigation :** Layout remains mounted. Uptime timer ticks continuously (12s, 13s, 14s...), and counter remains unchanged (5). Only the dynamic dynamic page slug header updates.
- **Root Scope Navigation:** The subtree unmounts completely. Navigating back to /blog resets Uptime Timer and State Counter to 0.

CONCLUSION:

Layout compositions using Next.js root layouts and localized client sub-layouts were successfully constructed. The experiment verified client-side partial rendering, tree reconciliation, state retention across nested sub-routes, and state reset behavior when leaving localized layout segment boundaries.

POST LAB QUESTION:

Q: What is the primary functional difference between layout.tsx **and** template.tsx **during client-side route transitions in Next.js?**

A: The primary difference lies in component persistence and state lifecycle during client-side navigation:

- layout.tsx wraps route segments and **persists across navigations** between sibling routes. React preserves the underlying DOM node and maintains all client-side React state (useState, useEffect, media stream instances) intact.
- template.tsx creates a **new component instance** on every route change. Next.js unmounts the previous template instance and mounts a fresh one, causing all React states to reset, useEffect hooks to re-run, and DOM nodes to be recreated every time the user navigates.